

Lab Assignment 1

RTL Design and Functional Synthesis using Verilog

Date of Submission: 6-2-2026

Tools Used:

- Xilinx Vivado
- Verilog HDL

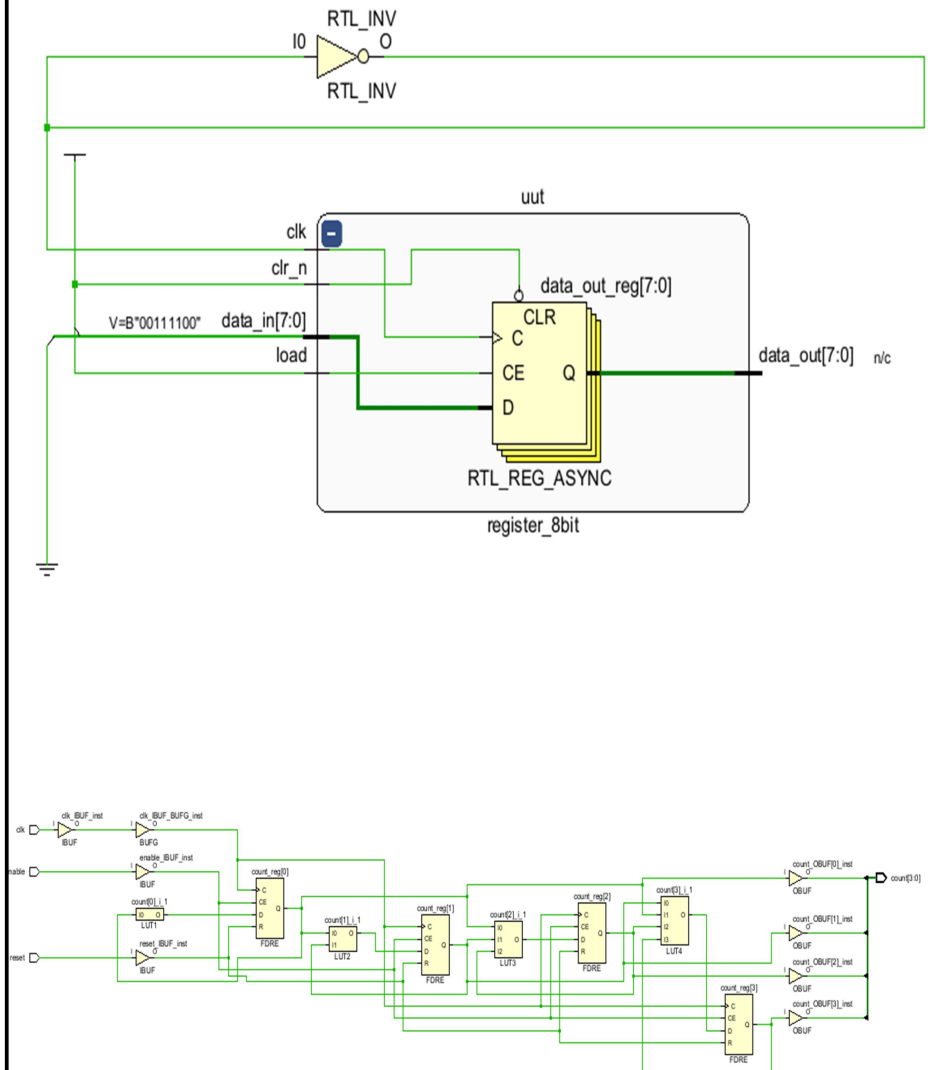
Problem 1: 4-bit Synchronous Up Counter																			
AIM:	Design a 4-bit synchronous up counter.																		
Specifications:	1. Active-high synchronous reset 2. Enable input 3. Counter increments on every rising edge of the clock when enable is high 4. Counter resets to zero when reset is asserted																		
Truth-Table:	Truth Table <table><tr><th>Clock ↑</th><th>Reset</th><th>Enable</th><th>Present Count</th></tr><tr><td>↑</td><td>1</td><td>X</td><td>XXXX</td></tr><tr><td>↑</td><td>0</td><td>0</td><td>Q</td></tr><tr><td>↑</td><td>0</td><td>1</td><td>Q</td></tr></table>			Clock ↑	Reset	Enable	Present Count	↑	1	X	XXXX	↑	0	0	Q	↑	0	1	Q
Clock ↑	Reset	Enable	Present Count																
↑	1	X	XXXX																
↑	0	0	Q																
↑	0	1	Q																
Verilog Code:	<pre>`timescale 1ns / 1ps module sync_up_counter_4bit (input clk, input reset, // Active-high synchronous reset input enable, output reg [3:0] count); always @(posedge clk) begin if (reset) count <= 4'b0000; else if (enable) count <= count + 1; else count <= count; end endmodule</pre>																		

RTL Schematic:	
Waveforms:	
Result/Conclusion:	The 4-bit synchronous up counter was successfully designed and verified using Verilog HDL. The simulation results confirm that the counter increments correctly when enable is high and resets synchronously when reset is asserted.

Problem 2: 8-bit Register with Load and Clear	
AIM:	Design an 8-bit register.
Specifications:	1. Load control signal 2. Active-low asynchronous clear 3. On load assertion, the register should load input data 4. Otherwise, the register should retain its previous value

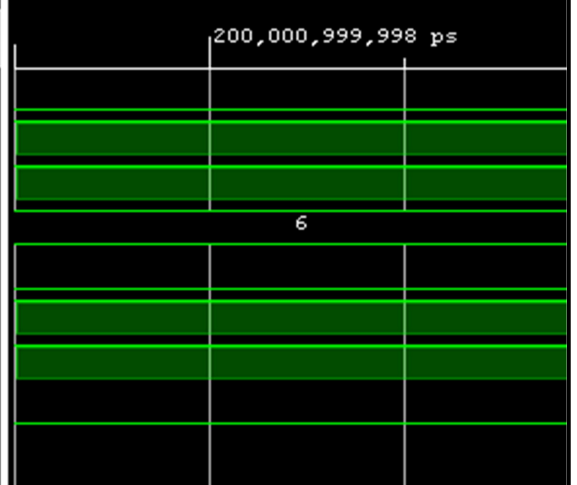
Truth-Table:	Truth Table <table><thead><tr><th>Clock ↑</th><th>Clear (active-low)</th><th>Load</th><th>Data In</th></tr></thead><tbody><tr><td>X</td><td>0</td><td>X</td><td>X</td></tr><tr><td>↑</td><td>1</td><td>1</td><td>D</td></tr><tr><td>↑</td><td>1</td><td>0</td><td>X</td></tr></tbody></table>	Clock ↑	Clear (active-low)	Load	Data In	X	0	X	X	↑	1	1	D	↑	1	0	X
Clock ↑	Clear (active-low)	Load	Data In														
X	0	X	X														
↑	1	1	D														
↑	1	0	X														
Verilog Code:	<pre>`timescale 1ns / 1ps module register_8bit (input clk, input clr_n, // Active-low asynchronous clear input load, input [7:0] data_in, output reg [7:0] data_out); always @(posedge clk or negedge clr_n) begin if (!clr_n) data_out <= 8'b00000000; else if (load) data_out <= data_in; else data_out <= data_out; end endmodule</pre>																

RTL Schematic:



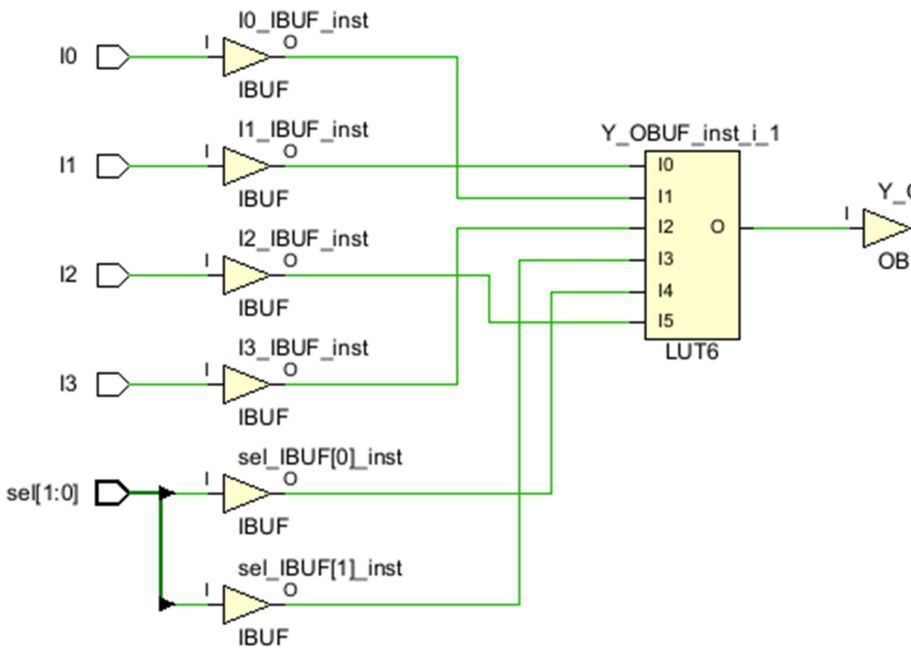
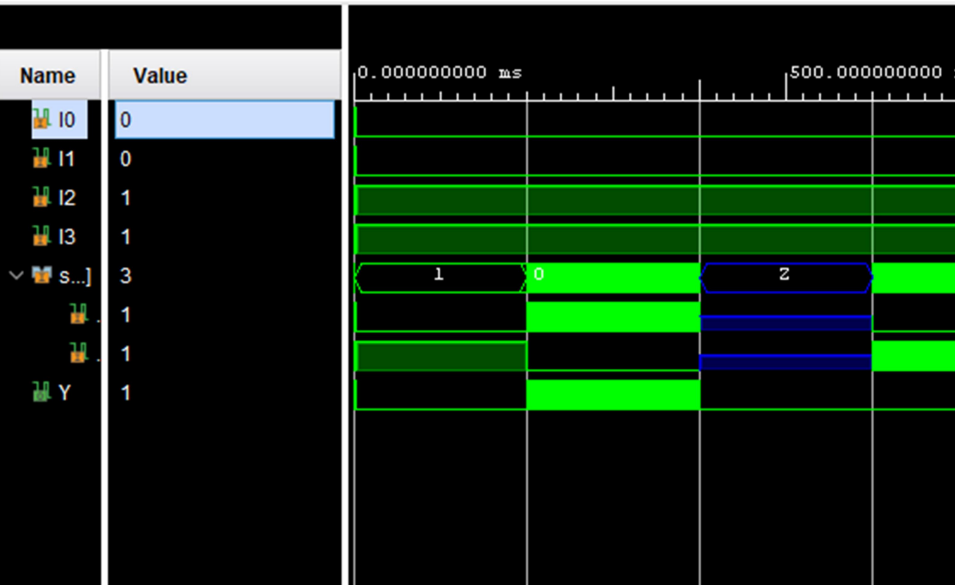
Waveforms:

Name	Value
clk	0
rese	1
e...e	1
c...	6
	0
	1
	1
	0



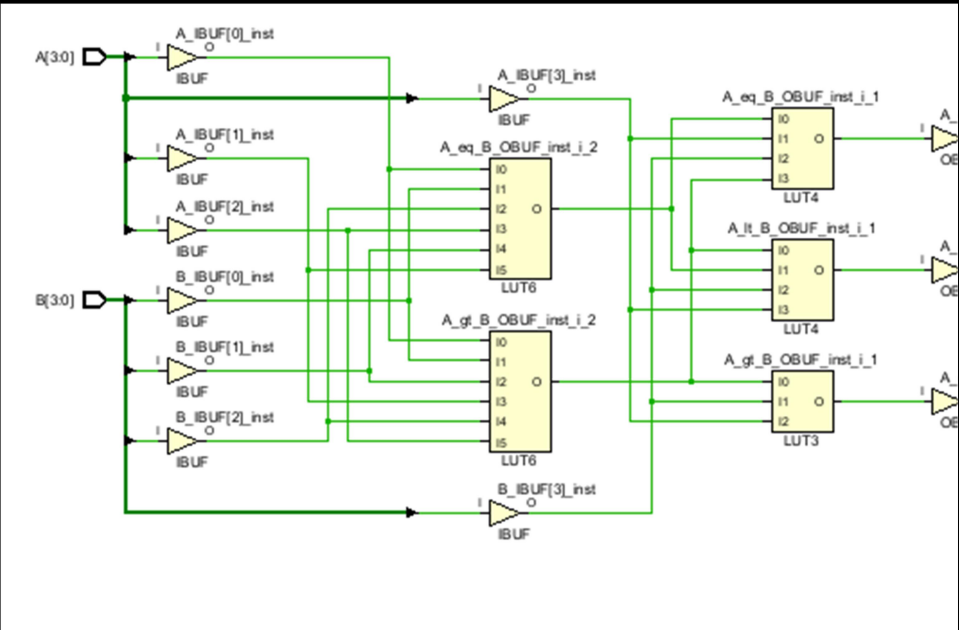
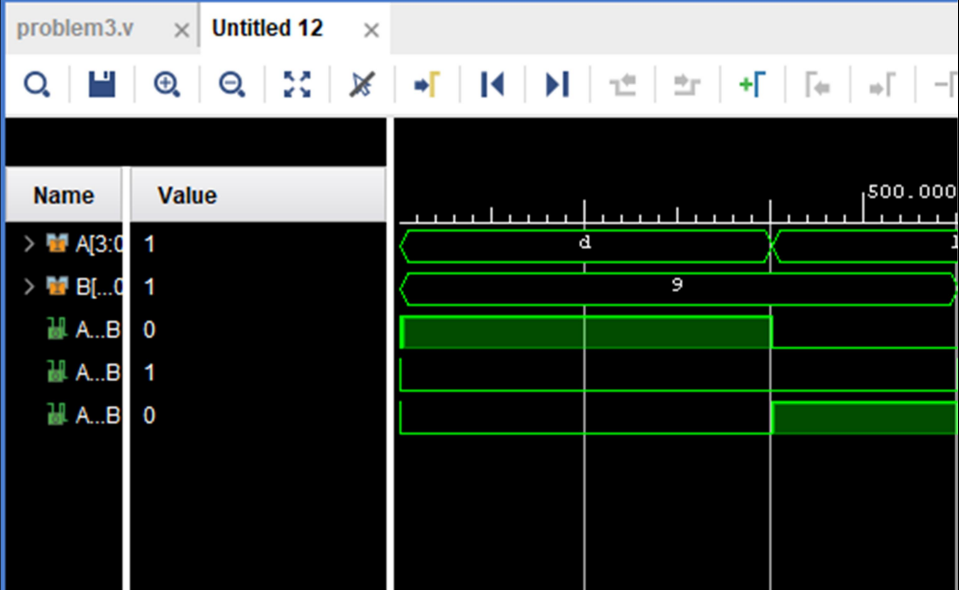
Result/Conclusion:	An 8-bit register with load and active-low asynchronous clear was successfully designed and verified using Verilog HDL. The simulation confirms correct loading, data retention, and immediate clearing functionality.
--------------------	---

Problem 3: 4:1 Multiplexer											
AIM:	Design a 4-to-1 multiplexer using a case statement										
Specifications:	1. Four single-bit inputs										
	2. Two-bit select line										
	3. One single-bit output										
Truth-Table:	Truth Table <table> <tr> <th>Select (S1 S0)</th><th>Output (Y)</th></tr> <tr> <td>00</td><td>I0</td></tr> <tr> <td>01</td><td>I1</td></tr> <tr> <td>10</td><td>I2</td></tr> <tr> <td>11</td><td>I3</td></tr> </table>	Select (S1 S0)	Output (Y)	00	I0	01	I1	10	I2	11	I3
Select (S1 S0)	Output (Y)										
00	I0										
01	I1										
10	I2										
11	I3										
Verilog Code:	<pre> module mux_4to1 (input I0, input I1, input I2, input I3, input [1:0] sel, output reg Y); always @(*) begin case (sel) 2'b00: Y = I0; 2'b01: Y = I1; 2'b10: Y = I2; 2'b11: Y = I3; default: Y = 1'b0; endcase end endmodule </pre>										

RTL Schematic:	
Waveforms:	
Result/Conclusion:	A 4-to-1 multiplexer was successfully designed using a case statement in Verilog HDL. Simulation results verify that the output correctly reflects the selected input for all select line combinations.

Problem 4: 4-bit Magnitude Comparator	
AIM:	Design a 4-bit magnitude comparator that compares two 4-bit numbers A and B

Specifications:	Comparator generates the following outputs: 1. $A > B$ 2. $A = B$ 3. $A < B$																
Truth-Table:	Truth Table <table><tr><th>A (4-bit)</th><th>B (4-bit)</th><th>A > B</th><th>A = B</th></tr><tr><td>A > B</td><td>B</td><td>1</td><td>0</td></tr><tr><td>A = B</td><td>B</td><td>0</td><td>1</td></tr><tr><td>A < B</td><td>B</td><td>0</td><td>0</td></tr></table>	A (4-bit)	B (4-bit)	A > B	A = B	A > B	B	1	0	A = B	B	0	1	A < B	B	0	0
A (4-bit)	B (4-bit)	A > B	A = B														
A > B	B	1	0														
A = B	B	0	1														
A < B	B	0	0														
Verilog Code:	<pre>module comparator_4bit (input [3:0] A, input [3:0] B, output reg A_gt_B, output reg A_eq_B, output reg A_lt_B); always @(*) begin if (A > B) begin A_gt_B = 1'b1; A_eq_B = 1'b0; A_lt_B = 1'b0; end else if (A == B) begin A_gt_B = 1'b0; A_eq_B = 1'b1; A_lt_B = 1'b0; end else begin A_gt_B = 1'b0; A_eq_B = 1'b0; A_lt_B = 1'b1; end end endmodule</pre>																

RTL Schematic:	
Waveforms:	
Result/Conclusion:	A 4-bit magnitude comparator was successfully designed and verified using Verilog HDL. The simulation results confirm correct comparison between inputs A and B for all conditions.

Problem 5: Mealy FSM – Sequence Detector (1011)	
AIM:	Design a Mealy finite state machine that detects the overlapping sequence 1011 from a serial input stream.
Specifications:	- Output should be asserted when the sequence is detected - FSM should allow overlapping sequences

Truth-Table:	<table><tr><th colspan="3">Truth Table (State Transition Table)</th></tr><tr><th>Present State</th><th>Input</th><th>Next State</th></tr><tr><td>S0</td><td>0</td><td>S0</td></tr><tr><td>S0</td><td>1</td><td>S1</td></tr><tr><td>S1</td><td>0</td><td>S2</td></tr><tr><td>S1</td><td>1</td><td>S1</td></tr><tr><td>S2</td><td>0</td><td>S0</td></tr><tr><td>S2</td><td>1</td><td>S3</td></tr><tr><td>S3</td><td>0</td><td>S2</td></tr><tr><td>S3</td><td>1</td><td>S1</td></tr></table>	Truth Table (State Transition Table)			Present State	Input	Next State	S0	0	S0	S0	1	S1	S1	0	S2	S1	1	S1	S2	0	S0	S2	1	S3	S3	0	S2	S3	1	S1
Truth Table (State Transition Table)																															
Present State	Input	Next State																													
S0	0	S0																													
S0	1	S1																													
S1	0	S2																													
S1	1	S1																													
S2	0	S0																													
S2	1	S3																													
S3	0	S2																													
S3	1	S1																													
Verilog Code:	<pre>module mealy_1011_detector (input clk, input reset, // Active-high synchronous reset input din, output reg dout); parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11; reg [1:0] state, next_state; // State register always @(posedge clk) begin if (reset) state <= S0; else state <= next_state; end // Next-state and output logic always @(*) begin dout = 1'b0; case (state) S0: begin if (din) next_state = S1; else next_state = S0; end S1: begin if (din) next_state = S1; else next_state = S2; end end</pre>																														

	<pre> S2: begin if (din) next_state = S3; else next_state = S0; end S3: begin if (din) begin next_state = S1; dout = 1'b1; // Sequence detected end else next_state = S2; end default: next_state = S0; endcase end endmodule </pre>																						
RTL Schematic:																							
Waveforms:	<table border="1"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr><td>clk</td><td>1</td></tr> <tr><td>rese</td><td>1</td></tr> <tr><td>din</td><td>0</td></tr> <tr><td>dout</td><td>0</td></tr> <tr><td>> st...</td><td>0</td></tr> <tr><td>> ne...</td><td>0</td></tr> <tr><td>> S0...</td><td>0</td></tr> <tr><td>> S1...</td><td>1</td></tr> <tr><td>> S2...</td><td>2</td></tr> <tr><td>> S3...</td><td>3</td></tr> </tbody> </table>	Name	Value	clk	1	rese	1	din	0	dout	0	> st...	0	> ne...	0	> S0...	0	> S1...	1	> S2...	2	> S3...	3
Name	Value																						
clk	1																						
rese	1																						
din	0																						
dout	0																						
> st...	0																						
> ne...	0																						
> S0...	0																						
> S1...	1																						
> S2...	2																						
> S3...	3																						
Result/Conclusion:	A Mealy FSM sequence detector for the overlapping sequence 1011 was successfully designed and verified using Verilog HDL. Simulation results confirm accurate detection and correct overlapping behavior.																						

Name of Student:	YASH IZATE	Name of Teacher	Dr.Ankita Tijare
Roll No:	69	Marks Out of 15	